

Java Fundamentals, OOP, Event Handling and Swing GUI

PART 1: FUNDAMENTALS

1. Introduction to Java

Java is a platform-independent, object-oriented language — “write once, run anywhere” thanks to the Java Virtual Machine (JVM), which runs compiled Java bytecode on any OS.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Real-world example: Android apps, enterprise banking systems, and large-scale backend services (like parts of Netflix’s infrastructure) run on Java because of its stability and portability.

2. Variables and Data Types

```
int age = 20;  
double gpa = 3.8;  
char grade = 'A';  
boolean isActive = true;  
String name = "Ahmad";
```

Type	Size	Example
int	4 bytes	int marks = 90;
double	8 bytes	double price = 9.99;
char	2 bytes	char c = 'A';
boolean	1 bit	boolean flag = true;

Real-world example: Same as the real world — a “price” naturally fits double, a “yes/no subscription status” fits boolean.

3. Control Flow

```
int marks = 85;  
if (marks >= 90) {  
    System.out.println("Grade A");  
} else if (marks >= 75) {  
    System.out.println("Grade B");  
} else {  
    System.out.println("Grade C");  
}
```

```

for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}

int count = 0;
while (count < 5) {
    System.out.println(count);
    count++;
}

```

Real-world example: A loop scanning through a list of orders to find ones marked “pending” is a standard for loop with an if check inside.

4. Methods

```

public class MathUtils {
    static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(add(5, 7));
    }
}

```

Real-world example: A calculateTax() method reused everywhere a bill needs tax applied, instead of repeating the formula.

5. Arrays

```

int[] marks = {90, 85, 78, 92};
for (int m : marks) {
    System.out.println(m);
}

```

Real-world example: Storing a week’s daily temperatures to compute a weekly average.

PART 2: OBJECT-ORIENTED PROGRAMMING

1. Classes and Objects

```

class Car {
    String brand;
    int speed;

    void accelerate() {
        speed += 10;
        System.out.println(brand + " speed is now " + speed);
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.brand = "Toyota";
    }
}

```

```
        myCar.speed = 0;
        myCar.accelerate();
    }
}
```

Real-world example: Car is the blueprint; each new Car() is a separate real car object with its own state.

2. Constructors

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
        System.out.println(name + " has enrolled.");
    }
}
```

Real-world example: When a new user signs up on an app, the constructor equivalent sets up their initial profile data the moment the account object is created.

3. Encapsulation

```
class BankAccount {
    private double balance;

    BankAccount(double initial) {
        balance = initial;
    }

    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    double getBalance() {
        return balance;
    }
}
```

Real-world example: Private balance + public deposit/getBalance methods = you can't directly overwrite an account's balance from outside, only interact through defined operations.

4. Inheritance

```
class Animal {
    String name;
    void eat() {
        System.out.println(name + " is eating.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println(name + " is barking.");
    }
}
```

Real-world example: SavingsAccount and CurrentAccount both extending a base BankAccount class to reuse deposit/withdraw logic.

5. Polymorphism

Overloading (compile-time):

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
```

Overriding (runtime):

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a circle");
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s = new Circle();
        s.draw(); // Drawing a circle
    }
}
```

Real-world example: A ride-hailing app calling calculateFare() on Bike, Car, or Premium objects — same method call, different fare logic per vehicle type.

6. Abstraction

```
abstract class PaymentMethod {
    abstract void pay(double amount);
}

class CreditCard extends PaymentMethod {
    void pay(double amount) {
        System.out.println("Paid " + amount + " using Credit Card.");
    }
}
```

Interfaces (pure abstraction, no implementation):

```
interface Payable {
    void pay(double amount);
}

class PayPal implements Payable {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using PayPal.");
    }
}
```

Real-world example: At checkout, you just see “Payment Successful” — the actual verification steps are abstracted away from you.

7. Static Members, Enums, Exception Handling (Quick Reference)

```
class Counter {
    static int count = 0;
    Counter() { count++; }
}

enum Day { MONDAY, TUESDAY, WEDNESDAY }

try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
} finally {
    System.out.println("Cleanup code runs here regardless.");
}
```

Real-world example: A file-reading operation wrapped in try-catch so the app shows “File not found” instead of crashing.

PART 3: EVENT HANDLING

Event handling is how a Java program responds to user actions (button clicks, key presses, mouse movement).

Core Concepts

- **Event Source** — the component that generates the event (e.g., a button)
- **Event Object** — carries information about the event (e.g., `ActionEvent`)
- **Event Listener** — an interface that “listens” for events and reacts

```
import java.awt.event.*;
import javax.swing.*;

public class ButtonExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Event Handling Demo");
        JButton button = new JButton("Click Me");

        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                System.out.println("Button was clicked!");
            }
        });

        frame.add(button);
        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

- `addActionListener` — registers a listener on the button
- `actionPerformed` — the method that runs when the event (click)

happens

Using lambda expressions (modern, cleaner syntax):

```
button.addActionListener(e -> System.out.println("Clicked via  
lambda!"));
```

Real-world example: Every time you click “Submit” on a login form, an event listener is what triggers the code that checks your username and password.

PART 4: JAVA SWING & GUI

Swing is Java’s toolkit for building desktop graphical user interfaces (windows, buttons, text fields, menus, etc.).

1. Basic Window (JFrame)

```
import javax.swing.*;

public class BasicWindow {
    public static void main(String[] args) {
        JFrame frame = new JFrame("My First GUI");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

- JFrame — the main window container
 - setDefaultCloseOperation — what happens when you click the X button
 - setVisible(true) — actually renders the window on screen
-

2. Common Swing Components

```
import javax.swing.*;
import java.awt.*;

public class SwingComponents {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Registration Form");
        frame.setLayout(new FlowLayout());

        JLabel nameLabel = new JLabel("Name:");
        JTextField nameField = new JTextField(15);

        JLabel passLabel = new JLabel("Password:");
        JPasswordField passField = new JPasswordField(15);

        JCheckBox termsCheck = new JCheckBox("I agree to terms");

        JRadioButton male = new JRadioButton("Male");
        JRadioButton female = new JRadioButton("Female");
        ButtonGroup genderGroup = new ButtonGroup();
        genderGroup.add(male);
        genderGroup.add(female);

        JButton submitButton = new JButton("Submit");
```

```

        frame.add(nameLabel);
        frame.add(nameField);
        frame.add(passLabel);
        frame.add(passField);
        frame.add(termsCheck);
        frame.add(male);
        frame.add(female);
        frame.add(submitButton);

        frame.setSize(350, 250);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

Component	Purpose
JLabel	display text
JTextField	single-line text input
JPasswordField	masked password input
JButton	clickable button
JCheckBox	toggle option
JRadioButton + ButtonGroup	pick one of several options

Real-world example: This is exactly the structure behind a real desktop registration form — labels next to input fields, a checkbox for terms, radio buttons for gender, and a submit button.

3. Layout Managers

Layout managers control how components are arranged inside a container.

```

line    frame.setLayout(new FlowLayout()); // left-to-right, wraps to next
Center  frame.setLayout(new BorderLayout()); // North, South, East, West,
        frame.setLayout(new GridLayout(2,2)); // fixed rows/columns grid

```

Real-world example: A calculator app uses GridLayout for its number buttons (evenly spaced rows/columns), while a settings page might use BorderLayout (menu on top, content in the center, footer at the bottom).

4. Connecting Events to Swing Components (Full Example)

```

import javax.swing.*;
import java.awt.event.*;

public class SimpleCalculator extends JFrame {
    JTextField num1Field, num2Field, resultField;

    public SimpleCalculator() {
        setTitle("Simple Calculator");
        setLayout(new java.awt.FlowLayout());

        num1Field = new JTextField(5);
        num2Field = new JTextField(5);
    }
}

```

```

        resultField = new JTextField(10);
        resultField.setEditable(false);

        JButton addButton = new JButton("Add");

        addButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                int num1 = Integer.parseInt(num1Field.getText());
                int num2 = Integer.parseInt(num2Field.getText());
                resultField.setText(String.valueOf(num1 + num2));
            }
        });

        add(num1Field);
        add(num2Field);
        add(addButton);
        add(resultField);

        setSize(300, 150);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public static void main(String[] args) {
        new SimpleCalculator();
    }
}

```

This ties everything together: **Swing components** build the interface, and **event handling** (`addActionListener`) makes it interactive — clicking “Add” reads the text fields, computes a result, and displays it.

Real-world example: This is the exact pattern behind your own Tkinter/Swing calculator projects — text fields for input, a button with an event listener, and a read-only field to display the result.

Quick Recap Table

Concept	One-line definition
Class/Object	Blueprint / instance
Encapsulation	Hiding data behind private + public methods
Inheritance	Reusing a parent class
Polymorphism	Same method, different behavior
Abstraction	Hiding complexity via abstract classes/interfaces
Event Listener	Code that reacts to user actions
JFrame	The main GUI window
Layout Manager	Controls how components are arranged

Notes by Ahmad — CodeNotes